

Agentic AI LAB INTERNAL

Experiments 1, 2 and 7 — Code and Output

Roll No.: 2311CS040145

Name: Racha Divyabhanu

Experiment 1: Text-to-SQL Workflow | Experiment 2: RAG-based Question Answering | Experiment 7: Deep Research Agent (Planning + Reflection)

Experiment 1: Text-to-SQL Workflow

Aim: Given a natural-language question and a SQLite database, automatically generate a correct SQL query, execute it, self-correct on failure, and synthesize a plain-English answer.

Pipeline stages implemented (file: `exp_text_to_sql.py`):

- Schema Introspection — SchemaExtractor reads table/column/foreign-key info via PRAGMA table_info and PRAGMA foreign_key_list.
- SQL Generation — SQLGenerator turns the question + schema into a SQL query via an LLM (OpenAI/Anthropic) or a rule-based fallback.
- Execution — QueryExecutor runs the SQL against SQLite and catches errors.
- Self-Correction Loop — on execution failure, the error is fed back to the generator to produce a corrected query, up to --max_retries times.
- Answer Synthesis — AnswerSynthesizer converts the raw result rows into a natural-language answer.

Source Code — `exp_text_to_sql.py`

`exp_text_to_sql.py`

```
"""
Experiment: Text-to-SQL Workflow
=====
Given a natural-language question and a database, produce a correct SQL
query, run it, and return the results in plain English.

Pipeline stages (mirrors the other experiments' style — each stage is its
own class/method so it's easy to inspect and grade):

1. SCHEMA INTROSPECTION -> SchemaExtractor (read table/column info from the DB)
2. SQL GENERATION        -> SQLGenerator   (NL question + schema -> SQL, via LLM)
3. EXECUTION             -> QueryExecutor  (run SQL against SQLite, catch errors)
4. SELF-CORRECTION LOOP  -> if execution fails, the error is fed back to the
                           LLM to regenerate a fixed query (up to N retries)
5. ANSWER SYNTHESIS      -> turn the raw SQL result rows into a natural-
                           language answer

LLM backend: OpenAI (OPENAI_API_KEY) or Anthropic (ANTHROPIC_API_KEY) if set.
Without either, a small rule-based fallback handles a handful of common
question patterns (counts, simple filters, "list all X") so the workflow is
still runnable end-to-end with zero setup — useful for testing the pipeline
control flow before you have an API key.

Comes with a script to build a sample SQLite database (`--init_sample_db`)
so you can try it immediately with no setup beyond `pip install`.
"""

import os
import re
import json
import sqlite3
import argparse
from dataclasses import dataclass, field
from typing import List, Dict, Any, Optional

# -----
# 0. Sample database (so this is runnable with zero setup)
# -----

def init_sample_db(db_path: str):
    """Creates a small company DB: departments + employees, with sample rows."""
```

```

if os.path.exists(db_path):
    os.remove(db_path)
conn = sqlite3.connect(db_path)
cur = conn.cursor()
cur.executescript("""
CREATE TABLE departments (
    dept_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    location TEXT
);

CREATE TABLE employees (
    emp_id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    dept_id INTEGER,
    salary REAL,
    hire_date TEXT,
    FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
""")
cur.executemany(
    "INSERT INTO departments (dept_id, name, location) VALUES (?, ?, ?)",
    [
        (1, "Engineering", "Bangalore"),
        (2, "Sales", "Mumbai"),
        (3, "HR", "Hyderabad"),
    ],
)
cur.executemany(
    "INSERT INTO employees (emp_id, name, dept_id, salary, hire_date) VALUES (?, ?, ?, ?, ?)",
    [
        (1, "Asha Rao", 1, 95000, "2021-03-01"),
        (2, "Vikram Singh", 1, 88000, "2022-07-15"),
        (3, "Meera Iyer", 2, 72000, "2020-11-20"),
        (4, "Rahul Nair", 3, 65000, "2023-01-10"),
        (5, "Priya Sharma", 1, 102000, "2019-06-05"),
        (6, "Karan Mehta", 2, 78000, "2022-02-28"),
    ],
)
conn.commit()
conn.close()
print(f"[init_sample_db] created sample DB at {db_path}")

# -----
# 1. Schema introspection
# -----

@dataclass
class TableSchema:
    name: str
    columns: List[Dict[str, str]] = field(default_factory=list) # [{"name": "...", "type": "...}]
    foreign_keys: List[Dict[str, str]] = field(default_factory=list)

class SchemaExtractor:
    def __init__(self, db_path: str):
        self.db_path = db_path

    def extract(self) -> List[TableSchema]:
        conn = sqlite3.connect(self.db_path)
        cur = conn.cursor()
        cur.execute("SELECT name FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%'")
        tables = [r[0] for r in cur.fetchall()]

        schemas = []
        for t in tables:
            cur.execute(f"PRAGMA table_info({t})")
            cols = [{"name": row[1], "type": row[2]} for row in cur.fetchall()]

```

```

        cur.execute(f"PRAGMA foreign_key_list({t})")
        fks = [{"column": row[3], "references_table": row[2], "references_column": row[4]}
               for row in cur.fetchall()]

        schemas.append(TableSchema(name=t, columns=cols, foreign_keys=fks))

    conn.close()
    return schemas

    @staticmethod
    def to_prompt_string(schemas: List[TableSchema]) -> str:
        lines = []
        for s in schemas:
            col_str = ", ".join(f"{c['name']} {c['type']}" for c in s.columns)
            lines.append(f"TABLE {s.name}({col_str})")
            for fk in s.foreign_keys:
                lines.append(f"-- FK: {s.name}.{fk['column']} -> {fk['references_table']}.{fk['references_column']}")
        return "\n".join(lines)

# -----
# LLM client (same pattern as the other experiments)
# -----

class LLMClient:
    def __init__(self):
        self.backend = "fallback"
        if os.environ.get("OPENAI_API_KEY"):
            self.backend = "openai"
        elif os.environ.get("ANTHROPIC_API_KEY"):
            self.backend = "anthropic"
        print(f"[LLMClient] using backend: {self.backend}")

    def complete(self, system: str, user: str, max_tokens: int = 400) -> str:
        if self.backend == "openai":
            from openai import OpenAI
            client = OpenAI()
            resp = client.chat.completions.create(
                model="gpt-4o-mini",
                messages=[{"role": "system", "content": system},
                          {"role": "user", "content": user}],
                temperature=0,
                max_tokens=max_tokens,
            )
            return resp.choices[0].message.content.strip()

        if self.backend == "anthropic":
            import anthropic
            client = anthropic.Anthropic()
            resp = client.messages.create(
                model="claude-sonnet-4-6",
                max_tokens=max_tokens,
                system=system,
                messages=[{"role": "user", "content": user}],
            )
            return resp.content[0].text.strip()

        return self._fallback_sql(user)

    @staticmethod
    def _fallback_sql(user: str) -> str:
        """Very small rule-based NL->SQL for common patterns, so the pipeline
        runs without any API key. Looks for the question inside `user`."""
        q_match = re.search(r"Question:\s*(.+)", user)
        question = (q_match.group(1) if q_match else user).lower()

```

```

# table guess: prefer a table name (singular or plural) that actually
# appears in the question; otherwise fall back to the first table
all_tables = re.findall(r"TABLE (\w+)", user)
table = all_tables[0] if all_tables else "employees"
for t in all_tables:
    if t.lower() in question or t.lower().rstrip("s") in question:
        table = t
        break

# column guess for aggregate questions: prefer a column that belongs
# to the chosen table and is mentioned in the question
table_cols_match = re.search(rf"TABLE {re.escape(table)}\(((^)*\))", user)
table_cols = []
if table_cols_match:
    table_cols = [c.split()[0] for c in table_cols_match.group(1).split(",")]
agg_col = next((c for c in table_cols if c.lower() in question), "")

if "how many" in question or "count" in question:
    return f"SELECT COUNT(*) FROM {table};"
if "average" in question or "avg" in question:
    return f"SELECT AVG({agg_col}) FROM {table};"
if "list all" in question or "show all" in question or re.search(r"\ball\b", question):
    return f"SELECT * FROM {table};"
return f"SELECT * FROM {table} LIMIT 10;"

# -----
# 2. SQL generation
# -----

class SQLGenerator:
    SYSTEM_PROMPT = (
        "You are an expert SQLite query writer. Given a database schema and a "
        "natural-language question, output ONLY a single valid SQLite SQL "
        "query that answers the question. No explanation, no markdown fences, "
        "no comments – just the raw SQL statement ending in a semicolon."
    )

    FIX_SYSTEM_PROMPT = (
        "You are an expert SQLite query writer fixing a broken query. Given "
        "the schema, the original question, the failed SQL, and the database "
        "error message, output ONLY a corrected single valid SQLite query "
        "(no explanation, no markdown fences)."
    )

    def __init__(self, llm: LLMClient):
        self.llm = llm

    @staticmethod
    def _clean(sql: str) -> str:
        sql = sql.strip()
        sql = re.sub(r"^```(sql)?", "", sql).strip()
        sql = re.sub(r"```$", "", sql).strip()
        return sql

    def generate(self, question: str, schema_str: str) -> str:
        user = f"Schema:\n{schema_str}\n\nQuestion: {question}\n\nSQL:"
        sql = self.llm.complete(self.SYSTEM_PROMPT, user)
        return self._clean(sql)

    def fix(self, question: str, schema_str: str, failed_sql: str, error: str) -> str:
        user = (
            f"Schema:\n{schema_str}\n\nQuestion: {question}\n\n"
            f"Failed SQL:\n{failed_sql}\n\nError:\n{error}\n\nCorrected SQL:"
        )
        sql = self.llm.complete(self.FIX_SYSTEM_PROMPT, user)
        return self._clean(sql)

```

```

# -----
# 3. Execution
# -----

class QueryExecutor:
    def __init__(self, db_path: str):
        self.db_path = db_path

    def run(self, sql: str) -> Dict[str, Any]:
        conn = sqlite3.connect(self.db_path)
        cur = conn.cursor()
        try:
            cur.execute(sql)
            if sql.strip().lower().startswith("select"):
                cols = [d[0] for d in cur.description] if cur.description else []
                rows = cur.fetchall()
                return {"ok": True, "columns": cols, "rows": rows}
            else:
                conn.commit()
                return {"ok": True, "columns": [], "rows": [], "rowcount": cur.rowcount}
        except Exception as e:
            return {"ok": False, "error": str(e)}
        finally:
            conn.close()

# -----
# 5. Answer synthesis
# -----

class AnswerSynthesizer:
    SYSTEM_PROMPT = (
        "You turn SQL query results into a short, clear natural-language "
        "answer to the original question. Be concise and factual."
    )

    def __init__(self, llm: LLMClient):
        self.llm = llm

    def synthesize(self, question: str, columns: List[str], rows: List[tuple]) -> str:
        if self.llm.backend == "fallback":
            if not rows:
                return "No matching rows were found."
            if len(rows) == 1 and len(rows[0]) == 1:
                return f"Answer: {rows[0][0]}"
            preview = "\n".join(str(r) for r in rows[:10])
            return f"Columns: {columns}\nRows ({len(rows)} total, showing up to 10):\n{preview}"

        result_str = f"Columns: {columns}\nRows: {rows[:20]}"
        user = f"Question: {question}\n\nSQL result:\n{result_str}\n\nAnswer in plain English:"
        return self.llm.complete(self.SYSTEM_PROMPT, user, max_tokens=300)

# -----
# Full pipeline
# -----

class TextToSQLPipeline:
    def __init__(self, db_path: str, max_retries: int = 2):
        self.db_path = db_path
        self.schema_extractor = SchemaExtractor(db_path)
        self.llm = LLMClient()
        self.generator = SQLGenerator(self.llm)
        self.executor = QueryExecutor(db_path)
        self.synthesizer = AnswerSynthesizer(self.llm)
        self.max_retries = max_retries

```

```

def ask(self, question: str) -> Dict[str, Any]:
    schemas = self.schema_extractor.extract()
    schema_str = SchemaExtractor.to_prompt_string(schemas)
    print("[Schema]\n" + schema_str)

    sql = self.generator.generate(question, schema_str)
    attempts = [{"sql": sql}]
    result = self.executor.run(sql)

    retries = 0
    while not result["ok"] and retries < self.max_retries:
        print(f"[Self-correction] attempt {retries+1}: query failed with: {result['error']}")
        sql = self.generator.fix(question, schema_str, sql, result["error"])
        attempts.append({"sql": sql})
        result = self.executor.run(sql)
        retries += 1

    if not result["ok"]:
        return {
            "question": question,
            "final_sql": sql,
            "success": False,
            "error": result["error"],
            "attempts": attempts,
            "answer": f"I couldn't produce a working query after {retries} retries. Last error:
{result['error']}",
        }

    answer = self.synthesizer.synthesize(question, result.get("columns", []), result.get("rows", []))
    return {
        "question": question,
        "final_sql": sql,
        "success": True,
        "columns": result.get("columns", []),
        "rows": result.get("rows", []),
        "attempts": attempts,
        "answer": answer,
    }

# -----
# CLI
# -----

def main():
    parser = argparse.ArgumentParser(description="Text-to-SQL workflow experiment")
    parser.add_argument("--db", type=str, default="./company.db", help="Path to SQLite DB file")
    parser.add_argument("--question", type=str, help="Natural-language question to answer")
    parser.add_argument("--init_sample_db", action="store_true", help="Create/overwrite a sample DB at --db and
exit")
    parser.add_argument("--max_retries", type=int, default=2, help="Self-correction retries on SQL error")
    args = parser.parse_args()

    if args.init_sample_db:
        init_sample_db(args.db)
        return

    if not args.question:
        parser.error("--question is required unless using --init_sample_db")

    if not os.path.exists(args.db):
        print(f"[warning] {args.db} not found – creating a sample DB there. "
            f"Run with --init_sample_db explicitly to control this.")
        init_sample_db(args.db)

    pipeline = TextToSQLPipeline(args.db, max_retries=args.max_retries)
    result = pipeline.ask(args.question)

```

```

print("\n=== Generated SQL ===")
print(result["final_sql"])

if result["success"]:
    print("\n=== Raw result ===")
    print("columns:", result["columns"])
    for row in result["rows"][:10]:
        print(" ", row)
else:
    print("\n=== Error ===")
    print(result["error"])

print("\n=== Answer ===")
print(result["answer"])

if __name__ == "__main__":
    main()

```

Output

Query 1: "How many employees are there?"

```

(venv) PS C:\Users\divya\Desktop\2311cs040145_LAB_INTERNAL> python exp_text_to_sql.py --db ./company.db --
question "How many employees are there?"
[LLMClient] using backend: fallback
[Schema]
TABLE departments(dept_id INTEGER, name TEXT, location TEXT)
TABLE employees(emp_id INTEGER, name TEXT, dept_id INTEGER, salary REAL, hire_date TEXT)
-- FK: employees.dept_id -> departments.dept_id

=== Generated SQL ===
SELECT COUNT(*) FROM employees;

=== Raw result ===
columns: ['COUNT(*)']
(6,)

=== Answer ===
Answer: 6

```

Query 2: "What is the average salary of employees?"

```

(venv) PS C:\Users\divya\Desktop\2311cs040145_LAB_INTERNAL> python exp_text_to_sql.py --db ./company.db --
question "What is the average salary of employees?"
[LLMClient] using backend: fallback
[Schema]
TABLE departments(dept_id INTEGER, name TEXT, location TEXT)
TABLE employees(emp_id INTEGER, name TEXT, dept_id INTEGER, salary REAL, hire_date TEXT)
-- FK: employees.dept_id -> departments.dept_id

=== Generated SQL ===
SELECT AVG(salary) FROM employees;

=== Raw result ===
columns: ['AVG(salary)']
(83333.33333333333,)

=== Answer ===
Answer: 83333.33333333333

```

Query 3: "List all departments"

```

(venv) PS C:\Users\divya\Desktop\2311cs040145_LAB_INTERNAL> python exp_text_to_sql.py --db ./company.db --
question "List all departments"
[LLMClient] using backend: fallback
[Schema]
TABLE departments(dept_id INTEGER, name TEXT, location TEXT)
TABLE employees(emp_id INTEGER, name TEXT, dept_id INTEGER, salary REAL, hire_date TEXT)
  -- FK: employees.dept_id -> departments.dept_id

=== Generated SQL ===
SELECT * FROM departments;

=== Raw result ===
columns: ['dept_id', 'name', 'location']
(1, 'Engineering', 'Bangalore')
(2, 'Sales', 'Mumbai')
(3, 'HR', 'Hyderabad')

=== Answer ===
Columns: ['dept_id', 'name', 'location']
Rows (3 total, showing up to 10):
(1, 'Engineering', 'Bangalore')
(2, 'Sales', 'Mumbai')
(3, 'HR', 'Hyderabad')

```

Experiment 2: RAG-based Question Answering

Aim: Implement the three classic Retrieval-Augmented Generation (RAG) stages — Indexing, Retrieval and Generation — as separate, inspectable components, and answer a query grounded in a local document collection.

Pipeline stages implemented (file: `exp2_rag_qa.py`):

- Indexing — DocumentIndexer loads .txt/.md files, splits them into overlapping chunks, embeds each chunk (sentence-transformers if available, else TF-IDF) and stores them in a vector index (FAISS if available, else NumPy brute-force cosine similarity).
- Retrieval — Retriever embeds the query and returns the top-k most similar chunks by cosine similarity.
- Generation — Generator builds a grounded prompt from the retrieved chunks and calls an LLM (OpenAI/Anthropic); without an API key it falls back to an extractive answer built from the best-scoring chunk.

Source Code — `exp2_rag_qa.py`

`exp2_rag_qa.py`

```
"""
Experiment 2: RAG-based Question Answering System
=====
Implements the three classic RAG stages as separate, inspectable components:

1. INDEXING   -> DocumentIndexer (load, chunk, embed, store)
2. RETRIEVAL  -> Retriever       (embed query, similarity search, top-k)
3. GENERATION -> Generator       (build prompt from context, call LLM)

Works with or without extra libraries installed:
- Embeddings: sentence-transformers if available, else TF-IDF (sklearn) fallback.
- Vector store: FAISS if available, else a plain NumPy cosine-similarity index.
- Generation: OpenAI API if OPENAI_API_KEY is set, else Anthropic if
  ANTHROPIC_API_KEY is set, else a simple extractive fallback (no key needed,
  so you can test indexing+retrieval end-to-end without any API key).

Author: generated for coursework use.
"""

import os
import re
import glob
import argparse
import pickle
from dataclasses import dataclass, field
from typing import List, Dict, Tuple

import numpy as np

# -----
# 0. Data structures
# -----

@dataclass
class Chunk:
    doc_id: str
    chunk_id: int
    text: str
    source: str = ""
```

```

# -----
# 1. Embedding backend (auto-selects best available)
# -----

class Embedder:
    """Wraps sentence-transformers (preferred) or TF-IDF (fallback)."""

    def __init__(self):
        try:
            from sentence_transformers import SentenceTransformer
            self.model = SentenceTransformer("all-MiniLM-L6-v2")
            self.mode = "sentence-transformers"
        except Exception:
            from sklearn.feature_extraction.text import TfidfVectorizer
            self.vectorizer = TfidfVectorizer(stop_words="english")
            self.mode = "tfidf"
            self._fitted = False

    def fit(self, texts: List[str]):
        """Only needed for TF-IDF mode; must be called once on the corpus."""
        if self.mode == "tfidf":
            self.vectorizer.fit(texts)
            self._fitted = True

    def encode(self, texts: List[str]) -> np.ndarray:
        if self.mode == "sentence-transformers":
            return self.model.encode(
                texts, convert_to_numpy=True, normalize_embeddings=True
            ).astype("float32")
        else:
            if not self._fitted:
                # Fit-on-the-fly fallback (not ideal, but keeps single-query calls working)
                self.fit(texts)
            vecs = self.vectorizer.transform(texts).toarray()
            norms = np.linalg.norm(vecs, axis=1, keepdims=True)
            norms[norms == 0] = 1.0
            return (vecs / norms).astype("float32")

# -----
# 2. INDEXING
# -----

class DocumentIndexer:
    """
    Loads raw documents, splits them into overlapping chunks, embeds each
    chunk, and stores everything in a vector index (FAISS if available,
    else a NumPy matrix used for brute-force cosine similarity).
    """

    def __init__(self, embedder: Embedder, chunk_size: int = 500, chunk_overlap: int = 100):
        self.embedder = embedder
        self.chunk_size = chunk_size
        self.chunk_overlap = chunk_overlap
        self.chunks: List[Chunk] = []
        self.embeddings: np.ndarray = None
        self._faiss_index = None
        self._use_faiss = False

    # ---- loading ----
    @staticmethod
    def load_documents(folder: str) -> Dict[str, str]:
        """Reads every .txt/.md file in `folder` into {filename: text}."""
        docs = {}
        for path in glob.glob(os.path.join(folder, "**", "*.txt"), recursive=True):
            if path.lower().endswith((".txt", ".md")):
                with open(path, "r", encoding="utf-8", errors="ignore") as f:
                    docs[os.path.basename(path)] = f.read()
        return docs

```

```

# ---- chunking ----
def _chunk_text(self, text: str) -> List[str]:
    """Simple sliding-window chunker over whitespace-split tokens."""
    words = re.split(r"\s+", text.strip())
    words = [w for w in words if w]
    chunks = []
    step = max(1, self.chunk_size - self.chunk_overlap)
    for start in range(0, len(words), step):
        piece = words[start:start + self.chunk_size]
        if not piece:
            break
        chunks.append(" ".join(piece))
        if start + self.chunk_size >= len(words):
            break
    return chunks or [text.strip()]

# ---- build index ----
def build(self, docs: Dict[str, str]):
    self.chunks = []
    for doc_id, text in docs.items():
        for i, ch_text in enumerate(self._chunk_text(text)):
            self.chunks.append(Chunk(doc_id=doc_id, chunk_id=i, text=ch_text, source=doc_id))

    if not self.chunks:
        raise ValueError("No chunks produced – check that the docs folder has .txt/.md files.")

    texts = [c.text for c in self.chunks]
    self.embedder.fit(texts) # no-op for sentence-transformers
    self.embeddings = self.embedder.encode(texts)

    try:
        import faiss
        dim = self.embeddings.shape[1]
        index = faiss.IndexFlatIP(dim) # inner product on normalized vecs = cosine sim
        index.add(self.embeddings)
        self._faiss_index = index
        self._use_faiss = True
    except Exception:
        self._use_faiss = False # numpy brute-force fallback

    print(f"[Indexing] {len(self.chunks)} chunks from {len(docs)} docs "
          f"| embedder={self.embedder.mode} | vector_store={'faiss' if self._use_faiss else 'numpy'}")

def save(self, path: str):
    with open(path, "wb") as f:
        pickle.dump({"chunks": self.chunks, "embeddings": self.embeddings}, f)

def load(self, path: str):
    with open(path, "rb") as f:
        data = pickle.load(f)
    self.chunks = data["chunks"]
    self.embeddings = data["embeddings"]

# -----
# 3. RETRIEVAL
# -----

class Retriever:
    def __init__(self, indexer: DocumentIndexer):
        self.indexer = indexer

    def retrieve(self, query: str, top_k: int = 4) -> List[Tuple[Chunk, float]]:
        q_vec = self.indexer.embedder.encode([query])

        if self.indexer._use_faiss:
            scores, idxs = self.indexer._faiss_index.search(q_vec, top_k)

```

```

        results = [(self.indexer.chunks[i], float(s))
                    for i, s in zip(idxs[0], scores[0]) if i != -1]
    else:
        sims = (self.indexer.embeddings @ q_vec[0]) # cosine sim (vectors are normalized)
        top_idx = np.argsort(-sims)[:top_k]
        results = [(self.indexer.chunks[i], float(sims[i])) for i in top_idx]

    return results

# -----
# 4. GENERATION
# -----

class Generator:
    """
    Builds a grounded prompt from retrieved chunks and calls an LLM.
    Falls back to a template-based extractive answer if no API key is set,
    so the pipeline is always runnable end-to-end.
    """

    SYSTEM_PROMPT = (
        "You are a precise question-answering assistant. Answer the user's "
        "question using ONLY the provided context. If the answer is not in "
        "the context, say you don't have enough information. Cite the source "
        "filename(s) you used in square brackets."
    )

    def __init__(self):
        self.backend = "extractive-fallback"
        if os.environ.get("OPENAI_API_KEY"):
            self.backend = "openai"
        elif os.environ.get("ANTHROPIC_API_KEY"):
            self.backend = "anthropic"

    def _build_prompt(self, query: str, contexts: List[Tuple[Chunk, float]]) -> str:
        ctx_block = "\n\n".join(
            f"[{c.source} #chunk{c.chunk_id}] {c.text}" for c, _ in contexts
        )
        return f"Context:\n{ctx_block}\n\nQuestion: {query}\n\nAnswer:"

    def generate(self, query: str, contexts: List[Tuple[Chunk, float]]) -> str:
        prompt = self._build_prompt(query, contexts)

        if self.backend == "openai":
            from openai import OpenAI
            client = OpenAI()
            resp = client.chat.completions.create(
                model="gpt-4o-mini",
                messages=[
                    {"role": "system", "content": self.SYSTEM_PROMPT},
                    {"role": "user", "content": prompt},
                ],
                temperature=0.2,
            )
            return resp.choices[0].message.content.strip()

        if self.backend == "anthropic":
            import anthropic
            client = anthropic.Anthropic()
            resp = client.messages.create(
                model="claude-sonnet-4-6",
                max_tokens=500,
                system=self.SYSTEM_PROMPT,
                messages=[{"role": "user", "content": prompt}],
            )
            return resp.content[0].text.strip()

# ---- extractive fallback: no API key needed ----

```

```

        best_chunk, best_score = contexts[0]
        sources = ", ".join(sorted({c.source for c, _ in contexts}))
        return (
            f"(No LLM API key found – showing extractive fallback answer.)\n"
            f"Most relevant passage (score={best_score:.3f}):\n"
            f"\n{best_chunk.text[:600]}\n"
            f"[sources: {sources}]"
        )

# -----
# 5. Full pipeline
# -----

class RAGPipeline:
    def __init__(self, chunk_size: int = 500, chunk_overlap: int = 100, top_k: int = 4):
        self.embedder = Embedder()
        self.indexer = DocumentIndexer(self.embedder, chunk_size, chunk_overlap)
        self.retriever = None
        self.generator = Generator()
        self.top_k = top_k

    def index(self, docs_folder: str):
        docs = DocumentIndexer.load_documents(docs_folder)
        if not docs:
            raise FileNotFoundError(f"No .txt/.md files found in {docs_folder}")
        self.indexer.build(docs)
        self.retriever = Retriever(self.indexer)

    def ask(self, query: str) -> Dict:
        if self.retriever is None:
            raise RuntimeError("Call .index(docs_folder) before .ask(query)")
        contexts = self.retriever.retrieve(query, top_k=self.top_k)
        answer = self.generator.generate(query, contexts)
        return {
            "query": query,
            "answer": answer,
            "retrieved": [
                {"source": c.source, "chunk_id": c.chunk_id, "score": s, "text": c.text}
                for c, s in contexts
            ],
        }

# -----
# CLI
# -----

def main():
    parser = argparse.ArgumentParser(description="Exp 2: RAG-based QA system")
    parser.add_argument("--docs", type=str, default="./docs", help="Folder of .txt/.md documents")
    parser.add_argument("--query", type=str, required=True, help="Question to ask")
    parser.add_argument("--top_k", type=int, default=4)
    parser.add_argument("--chunk_size", type=int, default=500)
    parser.add_argument("--chunk_overlap", type=int, default=100)
    args = parser.parse_args()

    pipeline = RAGPipeline(chunk_size=args.chunk_size, chunk_overlap=args.chunk_overlap, top_k=args.top_k)
    pipeline.index(args.docs)
    result = pipeline.ask(args.query)

    print("\n=== Retrieved chunks ===")
    for r in result["retrieved"]:
        print(f"- [{r['source']}] #{r['chunk_id']} score={r['score']:.3f}\n {r['text'][:150]}...")

    print("\n=== Answer ===")
    print(result["answer"])

```

```
if __name__ == "__main__":
    main()
```

Output

Query: "What are the three stages of a RAG pipeline?"

```
(venv) PS C:\Users\divya\Desktop\2311cs040145_LAB_INTERNAL> .\venv\Scripts\python.exe exp2_rag_qa.py --docs . -
-query "What are the three stages of a RAG pipeline?"
[Indexing] 361 chunks from 56 docs | embedder=tfidf | vector_store=numpy

=== Retrieved chunks ===
- [report.md #0] score=0.407
  [fallback draft] n RAG and How to Solve Them in Production | Unstr...2026) Apr 10, 2026 · Retrieval-augmented
generation, or RAG, is a patte...
- [rag_intro.txt #0] score=0.221
  Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with text generation.
Instead of relying only on knowledge sto...
- [readme.txt #0] score=0.033
  ISO SCHEMATRON 2010 XSLT implementation by Rick Jelliffe with assistance from members of Schematron-love-in
maillist.

=== Answer ===
(No LLM API key found – showing extractive fallback answer.)
Most relevant passage (score=0.407):
"[fallback draft] n RAG and How to Solve Them in Production | Unstr...2026) Apr 10, 2026 · Retrieval-augmented
generation, or RAG, is a pattern where an LLM generates an answer using text it retrieves from your data ...
Sub-question: What are the main approaches or components of RAG pipelines?
Sub-question: What are current limitations or open challenges in RAG pipelines?
- (What are the common challenges in implementing RAG? - Educative) Jul 1, 2025 · RAG pipelines introduce
latency at several points: vector retrieval, reranking, prompt formatting"
[sources: rag_intro.txt, random-uint-data.txt, readme.txt, report.md]
```

Experiment 7: Deep Research Agent (Planning + Reflection)

Aim: Build an agentic content-generation workflow around Planning and Reflection: decompose a topic into sub-questions, research each one, draft a report, critique the draft, and iteratively revise it.

Pipeline stages implemented (file: `exp7_deep_research_agent.py`, class `DeepResearchAgent`):

- Plan — the topic is decomposed into 4–6 research sub-questions (`agent.plan()`).
- Research — each sub-question is searched independently (web via `duckduckgo-search`, or a local `--corpus` folder as fallback) (`agent.research()`).
- Draft — a first report is synthesized from all gathered notes (`agent.draft()`).
- Reflect — the agent critiques its own draft and returns structured JSON feedback: `coverage_gaps`, `structure_issues`, `accuracy_flags`, `overall_verdict` (`agent.reflect()`).
- Revise — the draft is rewritten to resolve that feedback; steps 4–5 repeat for `--reflection_rounds` iterations (`agent.revise()`).
- Outputs — the final draft is written to `report.md`, and the full trace (plan, notes, every draft and reflection) is written to `trace.json`.

Source Code — `exp7_deep_research_agent.py`

`exp7_deep_research_agent.py`

```
"""
Experiment 7: Deep Research Agent Workflow
=====
An agentic content-generation workflow built around two extra steps beyond
plain "search then write": PLANNING and REFLECTION.

Pipeline:
1. PLAN      -> break the topic into a small set of research sub-questions
2. RESEARCH  -> gather information for each sub-question (web search if
                 available, else the user-supplied local corpus)
3. DRAFT     -> synthesize a first draft from all gathered notes
4. REFLECT   -> the agent critiques its own draft (coverage, accuracy,
                 structure, missing angles) and produces concrete feedback
5. REVISE    -> the draft is rewritten to address the feedback
Steps 4-5 repeat for `--reflection_rounds` iterations (default 2).
6. FINALIZE  -> return the last draft plus the full trace (plan, notes,
                 every reflection + revision) for transparency/grading.

LLM backend: OpenAI (OPENAI_API_KEY) or Anthropic (ANTHROPIC_API_KEY) if set.
If neither is set, a lightweight rule-based fallback is used for every step
so the *workflow/control-flow* can still be run, inspected and graded without
any API key. Set a key for real-quality output.

Web search: uses the `duckduckgo_search` package if installed (no API key
required). Falls back to a local corpus folder (--corpus) or to fabricated
placeholder notes if neither is available.
"""

import os
import json
import argparse
import textwrap
from dataclasses import dataclass, field
from typing import List, Dict

# -----
# LLM client abstraction
# -----
```

```

class LLMClient:
    def __init__(self):
        self.backend = "fallback"
        if os.environ.get("OPENAI_API_KEY"):
            self.backend = "openai"
        elif os.environ.get("ANTHROPIC_API_KEY"):
            self.backend = "anthropic"
        print(f"[LLMClient] using backend: {self.backend}")

    def complete(self, system: str, user: str, max_tokens: int = 800) -> str:
        if self.backend == "openai":
            from openai import OpenAI
            client = OpenAI()
            resp = client.chat.completions.create(
                model="gpt-4o-mini",
                messages=[{"role": "system", "content": system},
                          {"role": "user", "content": user}],
                temperature=0.4,
                max_tokens=max_tokens,
            )
            return resp.choices[0].message.content.strip()

        if self.backend == "anthropic":
            import anthropic
            client = anthropic.Anthropic()
            resp = client.messages.create(
                model="claude-sonnet-4-6",
                max_tokens=max_tokens,
                system=system,
                messages=[{"role": "user", "content": user}],
            )
            return resp.content[0].text.strip()

        # ---- rule-based fallback (no API key needed) ----
        return self._fallback(system, user)

    @staticmethod
    def _fallback(system: str, user: str) -> str:
        """Deterministic stand-in so the pipeline is runnable with zero setup.
        Not meant to produce high-quality prose -- it exists so the control
        flow (plan -> research -> draft -> reflect -> revise) can be tested."""
        if "sub-questions" in system.lower() or "plan" in system.lower():
            topic = user.split("Topic:")[1].strip()
            return json.dumps([
                f"What is {topic}?",
                f"Why does {topic} matter / what problem does it solve?",
                f"What are the main approaches or components of {topic}?",
                f"What are current limitations or open challenges in {topic}?",
                f"What are notable real-world examples or applications of {topic}?",
            ])
        if "critique" in system.lower() or "reflect" in system.lower():
            return json.dumps({
                "coverage_gaps": ["Add concrete examples.", "Add a discussion of limitations."],
                "structure_issues": ["Add clearer section headings."],
                "accuracy_flags": [],
                "overall_verdict": "Draft is a reasonable first pass but needs more depth and examples."
            })
        # draft / revise fallback: just stitch the notes together
        return "[fallback draft]\n" + user[-1500:]

# -----
# Research / search tool
# -----

class SearchTool:
    def __init__(self, corpus_folder: str = None):
        self.corpus_folder = corpus_folder

```

```

self.mode = "duckduckgo"
try:
    import duckduckgo_search # noqa: F401
except Exception:
    self.mode = "local" if corpus_folder else "stub"

def search(self, query: str, max_results: int = 4) -> List[Dict]:
    if self.mode == "duckduckgo":
        from duckduckgo_search import DDGS
        with DDGS() as ddgs:
            results = list(ddgs.text(query, max_results=max_results))
            return [{"title": r.get("title", ""), "snippet": r.get("body", ""), "url": r.get("href", "")}
                    for r in results]

    if self.mode == "local":
        import glob
        hits = []
        for path in glob.glob(os.path.join(self.corpus_folder, "**", "*.*"), recursive=True):
            if path.lower().endswith((".txt", ".md")):
                with open(path, "r", encoding="utf-8", errors="ignore") as f:
                    text = f.read()
                    if any(w.lower() in text.lower() for w in query.split()):
                        hits.append({"title": os.path.basename(path), "snippet": text[:500], "url": path})
        return hits[:max_results]

    # stub: no search backend and no corpus available
    return [{"title": "stub", "snippet": f"(No search backend available. Placeholder note for: {query})",
            "url": ""}]

# -----
# Agent
# -----

@dataclass
class AgentTrace:
    topic: str
    plan: List[str] = field(default_factory=list)
    notes: Dict[str, List[Dict]] = field(default_factory=dict)
    drafts: List[str] = field(default_factory=list)
    reflections: List[Dict] = field(default_factory=list)

class DeepResearchAgent:
    PLAN_SYSTEM = (
        "You are a research planning assistant. Given a topic, break it down "
        "into 4-6 focused sub-questions that together give thorough coverage "
        "of the topic for a written report. Respond ONLY as a JSON list of "
        "strings, nothing else."
    )
    DRAFT_SYSTEM = (
        "You are a research writer. Using ONLY the provided research notes, "
        "write a well-structured, informative report on the topic. Use clear "
        "section headings that map to the sub-questions. Be concrete and "
        "avoid filler sentences."
    )
    REFLECT_SYSTEM = (
        "You are a critical editor performing self-reflection on a draft. "
        "Identify: (1) coverage_gaps - topics/sub-questions inadequately "
        "covered, (2) structure_issues - organization/clarity problems, "
        "(3) accuracy_flags - claims that look unsupported by the notes, "
        "(4) overall_verdict - one sentence. Respond ONLY as JSON with those "
        "four keys (three of them lists of short strings, overall_verdict a string)."
    )
    REVISE_SYSTEM = (
        "You are a research writer revising a draft based on editorial "
        "feedback. Produce a complete, improved version of the report that "
        "addresses every point in the feedback. Keep the good parts, expand "
        "or fix the weak parts. Output the full revised report only."
    )

```

```

)

def __init__(self, llm: LLMClient, search: SearchTool, reflection_rounds: int = 2):
    self.llm = llm
    self.search = search
    self.reflection_rounds = reflection_rounds

# ---- 1. Planning ----
def plan(self, topic: str) -> List[str]:
    raw = self.llm.complete(self.PLAN_SYSTEM, f"Topic: {topic}")
    try:
        sub_qs = json.loads(raw)
        assert isinstance(sub_qs, list)
    except Exception:
        # be lenient if the LLM didn't return clean JSON
        sub_qs = [line.strip("-* ").strip() for line in raw.splitlines() if line.strip()]
    return sub_qs

# ---- 2. Research ----
def research(self, sub_questions: List[str]) -> Dict[str, List[Dict]]:
    notes = {}
    for q in sub_questions:
        notes[q] = self.search.search(q)
    return notes

# ---- 3. Draft ----
def draft(self, topic: str, notes: Dict[str, List[Dict]]) -> str:
    notes_block = ""
    for q, results in notes.items():
        notes_block += f"\nSub-question: {q}\n"
        for r in results:
            notes_block += f" - ({r.get('title','')} {r.get('snippet','')})\n"
    user = f"Topic: {topic}\n\nResearch notes:\n{notes_block}\n\nWrite the report now."
    return self.llm.complete(self.DRAFT_SYSTEM, user, max_tokens=1200)

# ---- 4. Reflect ----
def reflect(self, topic: str, draft_text: str) -> Dict:
    user = f"Topic: {topic}\n\nDraft:\n{draft_text}\n\nCritique this draft."
    raw = self.llm.complete(self.REFLECT_SYSTEM, user, max_tokens=500)
    try:
        return json.loads(raw)
    except Exception:
        return {"coverage_gaps": [], "structure_issues": [], "accuracy_flags": [],
                "overall_verdict": raw[:300]}

# ---- 5. Revise ----
def revise(self, topic: str, draft_text: str, feedback: Dict) -> str:
    user = (
        f"Topic: {topic}\n\nCurrent draft:\n{draft_text}\n\n"
        f"Editorial feedback (JSON):\n{json.dumps(feedback, indent=2)}\n\n"
        "Produce the revised report."
    )
    return self.llm.complete(self.REVISE_SYSTEM, user, max_tokens=1400)

# ---- Full workflow ----
def run(self, topic: str) -> AgentTrace:
    trace = AgentTrace(topic=topic)

    print("[1/5] Planning sub-questions...")
    trace.plan = self.plan(topic)
    for q in trace.plan:
        print(f"    - {q}")

    print("[2/5] Researching each sub-question...")
    trace.notes = self.research(trace.plan)

    print("[3/5] Writing first draft...")
    current_draft = self.draft(topic, trace.notes)
    trace.drafts.append(current_draft)

```

```

        for i in range(self.reflection_rounds):
            print(f"[4/5] Reflection round {i+1}/{self.reflection_rounds}...")
            feedback = self.reflect(topic, current_draft)
            trace.reflections.append(feedback)

            print(f"[5/5] Revising based on feedback (round {i+1})...")
            current_draft = self.revise(topic, current_draft, feedback)
            trace.drafts.append(current_draft)

        return trace

# -----
# CLI
# -----

def main():
    parser = argparse.ArgumentParser(description="Exp 7: Deep research agent (planning + reflection)")
    parser.add_argument("--topic", type=str, required=True, help="Research topic / content brief")
    parser.add_argument("--corpus", type=str, default=None, help="Optional local folder of .txt/.md docs to search instead of the web")
    parser.add_argument("--reflection_rounds", type=int, default=2)
    parser.add_argument("--out", type=str, default="report.md", help="Where to save the final report")
    parser.add_argument("--trace_out", type=str, default="trace.json", help="Where to save the full run trace")
    args = parser.parse_args()

    llm = LLMClient()
    search = SearchTool(corpus_folder=args.corpus)
    agent = DeepResearchAgent(llm, search, reflection_rounds=args.reflection_rounds)

    trace = agent.run(args.topic)

    with open(args.out, "w", encoding="utf-8") as f:
        f.write(trace.drafts[-1])

    with open(args.trace_out, "w", encoding="utf-8") as f:
        json.dump({
            "topic": trace.topic,
            "plan": trace.plan,
            "notes": trace.notes,
            "num_drafts": len(trace.drafts),
            "reflections": trace.reflections,
            "drafts": trace.drafts,
        }, f, indent=2)

    print("\n=== FINAL REPORT (also saved to {}) ===\n".format(args.out))
    print(textwrap.shorten(trace.drafts[-1], width=1500, placeholder=" ...[truncated, see file]"))
    print(f"\nFull trace saved to {args.trace_out}")

if __name__ == "__main__":
    main()

```

Output

Command: `python exp7_deep_research_agent.py --topic "RAG pipelines" --corpus . --reflection_rounds 2`

```

(venv) PS C:\Users\divya\Desktop\2311cs040145_LAB_INTERNAL> .\venv\Scripts\python.exe
exp7_deep_research_agent.py --topic "RAG pipelines" --corpus . --reflection_rounds 2
[LLMClient] using backend: fallback
[1/5] Planning sub-questions...
- What is RAG pipelines?
- Why does RAG pipelines matter / what problem does it solve?

```

- What are the main approaches or components of RAG pipelines?
- What are current limitations or open challenges in RAG pipelines?
- What are notable real-world examples or applications of RAG pipelines?

[2/5] Researching each sub-question...

C:\Users\divya\Desktop\2311cs040145_LAB_INTERNAL\exp7_deep_research_agent.py:118: RuntimeWarning: This package (`duckduckgo\_search`) has been renamed to `ddgs`! Use `pip install ddgs` instead.

Screenshots

The screenshots below are terminal captures of the three experiments actually being run in VS Code on the student's machine.

```
Problems Output Debug Console Terminal Ports
(venv) PS C:\Users\divya\Desktop\2311cs040145_LAB_INTERNAL> cd "c:\Users\divya\Desktop\2311cs040145_LAB_INTERNAL"
>> .\venv\Scripts\python.exe exp2_rag_qa.py --docs . --query "What are the three stages of a RAG pipeline?"
[Indexing] 361 chunks from 56 docs | embedder=tfidf | vector_store=numpy

=== Retrieved chunks ===
- [report.md #0] score=0.407
  [fallback draft] n RAG and How to Solve Them in Production | Unstr\\(u0206) Apr 10, 2026 \\(u00b7 Retrieval-augment
ed generation, or RAG, is a patte...
- [rag_intro.txt #0] score=0.221
  Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with text generation. Instead
of relying only on knowledge sto...
- [readme.txt #0] score=0.033
  ISO SCHEMATRON 2010 XSLT implementation by Rick Jelliffe with assistance from members of Schematron-love-in maillist.
```

Fig 1 — Experiment 2 (RAG QA): indexing 361 chunks from 56 docs and retrieving the top-scoring chunks for the query "What are the three stages of a RAG pipeline?"

```
Problems Output Debug Console Terminal Ports
13 92 9 92 83 69 91 65 92 29 63 4...

=== Answer ===
(No LLM API key found - showing extractive fallback answer.)
Most relevant passage (score=0.407):
"[fallback draft] n RAG and How to Solve Them in Production | Unstr\\\\\\\\u2026 Apr 10, 2026 \\\\u00b7 Retrieval-augmented
d generation, or RAG, is a pattern where an LLM generates an answer using text it retrieves from your data \\\\u2026\\\\\\
n\\\\\\\\\\nSub-question: What are the main approaches or components of RAG pipelines?\\\\\\\\\\n\\\\\\\\\\nSub-question: What are curre
nt limitations or open challenges in RAG pipelines?\\\\\\\\\\n - (What are the common challenges in implementing RAG? - Educa
tive) Jul 1, 2025 \\\\u00b7 RAG pipelines introduce latency at several points: vector retrieval, reranking, prompt form
ating"
[sources: rag_intro.txt, random-uint-data.txt, readme.txt, report.md]
(venv) PS C:\Users\divya\Desktop\2311cs040145_LAB_INTERNAL>
```

Fig 2 — Experiment 2 (RAG QA): generated (extractive fallback) answer with cited sources.

```
Problems Output Debug Console Terminal Ports
attning"
[sources: rag_intro.txt, random-uint-data.txt, readme.txt, rcd "c:\Users\diviya\Desktop\2311cs040145_LAB_INTERNAL"
>> .\venv\Scripts\python.exe exp7_deep_research_agent.py --topic "RAG pipelines" --corpus . --reflection_rounds 2
[LLMClient] using backend: fallback
[1/5] Planning sub-questions...
- What is RAG pipelines?
- Why does RAG pipelines matter / what problem does it solve?
- What are the main approaches or components of RAG pipelines?
- What are current limitations or open challenges in RAG pipelines?
- What are notable real-world examples or applications of RAG pipelines?
[2/5] Researching each sub-question...
C:\Users\diviya\Desktop\2311cs040145_LAB_INTERNAL\exp7_deep_research_agent.py:118: RuntimeWarning: This package (duckdu
ckgo_search) has been renamed to `ddgs`. Use `pip install ddgs` instead.
```

Fig 3 — Experiment 7 (Deep Research Agent): planning stage decomposing the topic "RAG pipelines" into sub-questions, followed by the research stage.

```

(venv) PS C:\Users\divya\Desktop\2311cs040145_LAB_INTERNAL> python exp_text_to_sql.py --db ./company.db --question "How
many employees are there?"
[LLMClient] using backend: fallback
[Schema]
TABLE departments(dept_id INTEGER, name TEXT, location TEXT)
TABLE employees(emp_id INTEGER, name TEXT, dept_id INTEGER, salary REAL, hire_date TEXT)
-- FK: employees.dept_id -> departments.dept_id

=== Generated SQL ===
SELECT COUNT(*) FROM employees;

=== Raw result ===
columns: ['COUNT(*)']

```

Fig 4 — Experiment 1 (Text-to-SQL): schema introspection, generated SQL, and result for "How many employees are there?"

```

=== Answer ===
Answer: 6
(venv) PS C:\Users\divya\Desktop\2311cs040145_LAB_INTERNAL> python exp_text_to_sql.py --db ./company.db --question "Wha
t is the average salary of employees?"
[LLMClient] using backend: fallback
[Schema]
TABLE departments(dept_id INTEGER, name TEXT, location TEXT)
TABLE employees(emp_id INTEGER, name TEXT, dept_id INTEGER, salary REAL, hire_date TEXT)
-- FK: employees.dept_id -> departments.dept_id

=== Generated SQL ===
SELECT AVG(salary) FROM employees;

=== Raw result ===
columns: ['AVG(salary)']
(83333.33333333333,)

=== Answer ===
Answer: 83333.33333333333
(venv) PS C:\Users\divya\Desktop\2311cs040145_LAB_INTERNAL> pip install duckduckgo-search
Requirement already satisfied: duckduckgo-search in c:\users\divya\desktop\2311cs040145_lab_internal\venv\lib\site-pack
ages (8.1.1)
Requirement already satisfied: click>=8.1.8 in c:\users\divya\desktop\2311cs040145_lab_internal\venv\lib\site-packages
(from duckduckgo-search) (8.5.0)
Requirement already satisfied: primp>=0.15.0 in c:\users\divya\desktop\2311cs040145_lab_internal\venv\lib\site-packages

```

Fig 5 — Experiment 1 (Text-to-SQL): schema, generated SQL, and result for "What is the average salary of employees?"

```

=== Raw result ===
columns: ['AVG(salary)']
(83333.33333333333,)

=== Answer ===
Answer: 83333.33333333333
(venv) PS C:\Users\divya\Desktop\2311cs040145_LAB_INTERNAL> pip install duckduckgo-search
Requirement already satisfied: duckduckgo-search in c:\users\divya\desktop\2311cs040145_lab_internal\venv\lib\site-pack
ages (8.1.1)
Requirement already satisfied: click>=8.1.8 in c:\users\divya\desktop\2311cs040145_lab_internal\venv\lib\site-packages
(from duckduckgo-search) (8.5.0)
Requirement already satisfied: primp>=0.15.0 in c:\users\divya\desktop\2311cs040145_lab_internal\venv\lib\site-packages

```

Fig 6 — Experiment 1 (Text-to-SQL): raw query result and synthesized answer.